

1. Introduction to Java

Java is a high-level, object-oriented programming language used to create software and applications. It was developed at Sun Microsystems and was first released in 1995.

Java is known for portability. Java source code is compiled into **bytecode**, which can run on different operating systems through the Java Virtual Machine (JVM).

Simple definition: Java is a programming language that allows developers to write applications that can run on many different platforms.

Why beginners learn Java: It teaches programming fundamentals and object-oriented programming clearly, and many Java concepts transfer to other languages.

A simple Java program

```
class Hello {  
    public static void main(String[] args) {  
        System.out.println("Hello, World!");  
    }  
}
```

The **main()** method is the usual starting point of a basic Java application. **System.out.println()** displays text on the screen.

2. Uses of Java

Java is a general-purpose language, so it is used in many areas of software development.

- **Web and backend applications:** Java is widely used for server-side software and enterprise systems.
- **Android development:** Java has historically been one of the major languages used for Android applications.
- **Enterprise software:** Organizations use Java for business applications, services and internal systems.
- **Desktop applications:** Java can be used to create graphical desktop programs.
- **Cloud and distributed systems:** Java is used in many server and cloud-based applications.
- **Data and big-data tools:** Several well-known data technologies have been built with or around the Java ecosystem.
- **Education:** Java is commonly used to teach programming, data structures and object-oriented concepts.

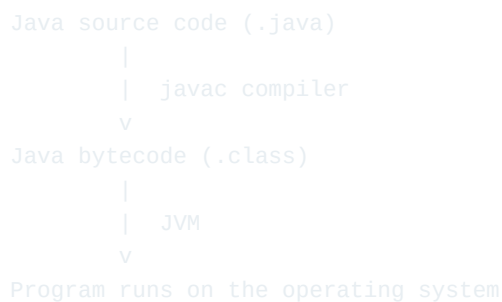
Important: Java is not limited to one type of application. Its libraries, tools and large ecosystem make it useful across many software-development tasks.

3. JDK, JRE and JVM

These three terms are important when starting Java.

- **JVM (Java Virtual Machine):** Executes Java bytecode and provides the environment that allows Java programs to run on a particular platform.
- **JRE (Java Runtime Environment):** Provides the components needed to run Java applications, including the JVM and supporting libraries.
- **JDK (Java Development Kit):** Provides tools needed to develop Java programs, including the Java compiler and runtime components.

Think of it like this:



Write once, run anywhere is a common phrase associated with Java's portability. In practice, the appropriate Java runtime/JVM must exist for the target platform.

4. Structure of a Java Program

A beginner should understand what the major parts of a Java program do before trying to memorize syntax.

```
public class Student {  
    public static void main(String[] args) {  
        int marks = 85;  
        System.out.println("Marks = " + marks);  
    }  
}
```

- **class Student:** Defines a class named Student.
- **main():** The entry point for this basic application.
- **int marks = 85:** Creates an integer variable and stores a value.
- **System.out.println():** Prints information to the console.
- **Semicolon (;):** Usually marks the end of a Java statement.

5. Variables and Data Types

A **variable** is a named location used to store a value. Every variable has a data type, a name and a value.

Java's eight primitive data types are **byte**, **short**, **int**, **long**, **float**, **double**, **char** and **boolean**.

- **byte**, **short**, **int**, **long**: Whole-number types.
- **float**, **double**: Decimal-number types.
- **char**: A single character, such as 'A'.
- **boolean**: Either true or false.

Example:

```
int age = 19;  
double height = 175.5;  
char grade = 'A';  
boolean passed = true;  
String name = "Ravi";
```

String is not a primitive data type; it is a class used to represent text.

6. Operators

Operators are symbols used to perform operations on values.

- **Arithmetic:** +, -, *, /, % — used for calculations.
- **Relational:** ==, !=, >, <, >=, <= — used to compare values.
- **Logical:** &&, ||, ! — used to combine or reverse conditions.
- **Assignment:** =, +=, -=, *=, /= — used to assign or update values.
- **Increment/Decrement:** ++ and -- — increase or decrease a value by one.

Example:

```
int a = 10;
int b = 3;

System.out.println(a + b); // 13
System.out.println(a % b); // 1
System.out.println(a > b); // true
```

7. Input and Output

Programs often need to receive data from the user and then display a result.

Output: Java commonly uses `System.out.println()` to display text or values.

Input: The `Scanner` class can read values entered through the keyboard.

Example:

```
import java.util.Scanner;

Scanner sc = new Scanner(System.in);

System.out.print("Enter your age: ");
int age = sc.nextInt();

System.out.println("Your age is " + age);
```

The `import` statement makes the `Scanner` class available. `nextInt()` reads an integer entered by the user.

8. Decision Making

Decision-making statements allow a program to choose what to do based on a condition.

if: Runs a block when a condition is true.

if-else: Chooses between two alternatives.

else-if: Checks multiple conditions.

switch: Selects one case from several possible values.

Example:

```
int marks = 75;

if (marks >= 40) {
    System.out.println("Pass");
} else {
    System.out.println("Fail");
}
```

9. Loops

A loop repeats a block of code while a condition or rule is satisfied.

- **for loop:** Useful when the number of repetitions is known or controlled by a counter.
- **while loop:** Repeats while a condition remains true.
- **do-while loop:** Executes the block at least once before checking the condition.

Example:

```
for (int i = 1; i <= 5; i++) {  
    System.out.println(i);  
}
```

10. Arrays and Strings

Array: An array stores multiple values of the same type under one variable name. Java array indexes normally start at 0.

```
int[] marks = {80, 75, 90, 85};

System.out.println(marks[0]); // 80
System.out.println(marks.length); // 4
```

String: A String stores text. Strings are objects in Java.

```
String name = "Ravi";

System.out.println(name);
System.out.println(name.length());
System.out.println(name.toUpperCase());
```

11. Methods

A **method** is a reusable block of code that performs a particular task. Methods reduce repetition and make programs easier to organize.

```
static int add(int a, int b) {  
    return a + b;  
}  
  
public static void main(String[] args) {  
    int result = add(10, 20);  
    System.out.println(result);  
}
```

- **Parameters:** Values received by the method.
- **Return value:** The result sent back by the method.
- **void:** Indicates that a method does not return a value.

12. Object-Oriented Programming (OOP)

Java is an object-oriented programming language. OOP organizes programs around **classes and objects**.

Class: A blueprint that describes data and behavior.

Object: An actual instance created from a class.

```
class Student {
    String name;
    int age;

    void display() {
        System.out.println(name + " - " + age);
    }
}

Student s1 = new Student();
s1.name = "Ravi";
s1.age = 19;
s1.display();
```

13. Constructors and Inheritance

Constructor: A special member used when an object is created. It has the same name as its class and does not have a return type.

```
class Student {
    String name;

    Student(String name) {
        this.name = name;
    }
}

Student s = new Student("Ravi");
```

Inheritance: Allows a child class to reuse accessible properties and methods of a parent class. Java commonly uses the **extends** keyword for class inheritance.

```
class Animal {
    void sound() {
        System.out.println("Animal sound");
    }
}

class Dog extends Animal {
    void bark() {
        System.out.println("Bark");
    }
}
```

14. The Four Main OOP Concepts

- **Encapsulation:** Keeping data and the methods that operate on it together, while controlling access to internal data.
- **Inheritance:** Building a new class from an existing class to reuse behavior.
- **Polymorphism:** Allowing the same method or interface concept to behave differently depending on the object or context.
- **Abstraction:** Showing essential features while hiding unnecessary implementation details.

These four concepts are central to understanding Java beyond basic syntax. They help developers build programs that are organized, reusable and easier to maintain.

15. Exception Handling

An **exception** is an event that can interrupt the normal flow of a program. Java provides exception-handling mechanisms so programs can respond to many runtime problems.

Important keywords include **try**, **catch**, **finally**, **throw** and **throws**.

```
try {  
    int result = 10 / 0;  
    System.out.println(result);  
} catch (ArithmeticException e) {  
    System.out.println("Cannot divide by zero.");  
}
```

The code that may cause an exception is placed in **try**. A matching **catch** block can handle the exception.

16. Advantages and Disadvantages

Advantages

- Platform independent through the JVM approach.
- Object-oriented design supports reusable and organized code.
- Automatic garbage collection reduces manual memory-management work.
- Large ecosystem of libraries, frameworks and development tools.
- Strong community support and extensive learning resources.
- Useful for many types of software, from backend systems to enterprise applications.

Disadvantages

- Java applications can use more memory than some lower-level alternatives.
- The syntax can feel verbose for small programs.
- Startup time can matter for some small or short-lived applications.
- Understanding the full ecosystem can be overwhelming for beginners.
- Java is not always the best fit for every task; language choice depends on project requirements.